

Modelado del movimiento de vibración libre mediante métodos de interpolación numérica

Adrián Castro¹ Juan Vargas² Esteban Alfaro³ Gabriel Fernandez⁴
Andrés Zamora⁵

^{1,2,3,4,5} Escuela de Ingeniería Civil, Universidad de Costa Rica

Curso MA-1006: Introducción al Análisis Numérico

30 de junio de 2026

Índice

1. Introducción	5
2. Planteamiento del problema	6
3. Objetivos	6
3.1. Objetivo general	6
3.2. Objetivos específicos	6
4. Marco teórico	7
4.1. Vibración libre en sistemas dinámicos	7
4.2. Representación discreta del movimiento	8
4.3. Interpolación polinómica de Lagrange	8
4.4. Splines cúbicos	9
4.5. Interpolación de Hermite	9
4.6. Criterios de comparación entre métodos	10
4.7. Método de comparación de resultados por análisis MAE y MSRT	10
5. Metodología	11
5.1. Descripción general de la base de datos	11
5.2. Preparación de los datos	11
5.3. Aplicación de la interpolación de Lagrange	11
5.4. Aplicación de splines cúbicos	12
5.5. Aplicación de la interpolación de Hermite	12
5.6. Comparación de resultados	12
5.7. Implementación computacional en MATLAB	12
6. Resultados esperados	13
7. Resultados obtenidos	14
7.1. Ecuación teórica	14
7.2. Polinomio interpolador de Hermite	16
7.3. Resultados obtenidos con el método de splines cúbicos	19
7.4. Polinomio interpolador de Lagrange	24
8. Análisis de resultados	27
9. Comparación cuantitativa de los métodos	33
10. Conclusiones	33

Índice de figuras

1.	Gráfico resultado de los datos obtenidos por la mesa vibratoria	13
2.	Gráfico experimental vs gráfico teórico	16
3.	Comparación del polinomio interpolador de Hermite con el movimiento de la estructura	19
4.	Comparación entre los datos originales de vibración libre y la aproximación mediante spline cúbico natural.	22
5.	Primeros coeficientes calculados para los tramos del spline cúbico natural.	23
6.	Comparación del polinomio interpolador de Lagrange con el movimiento de la estructura	27
7.	Comparación de los errores y tiempos de ejecución de cada método	31
8.	Comparación de los errores de cada método	32

Índice de cuadros

1. Métricas obtenidas en la comparación de métodos del archivo MATLAB Live Script. 33

1. Introducción

El estudio de las vibraciones es fundamental en la ingeniería civil y en la dinámica estructural. Las estructuras reales, como edificios, puentes, torres y elementos mecánicos, están sometidas a cargas dinámicas producidas por sismos, viento, tránsito o maquinaria. Estas acciones pueden generar movimientos oscilatorios que deben analizarse con cuidado para garantizar la seguridad y el buen desempeño de la estructura [3, 4]. Conocer la forma en que vibra un sistema permite estimar desplazamientos máximos, identificar frecuencias naturales y prevenir fenómenos no deseados, como la resonancia.

Dentro de este campo, un caso de especial interés es la vibración libre. Este tipo de movimiento ocurre cuando un sistema dinámico se separa de su posición de equilibrio y luego se deja oscilar sin la acción de fuerzas externas. La respuesta en vibración libre permite observar propiedades propias del sistema, como su frecuencia natural y su nivel de amortiguamiento. Estas propiedades no dependen directamente de la excitación aplicada, por lo que pueden entenderse como una huella dinámica del sistema estructural [3, 4].

Desde el punto de vista teórico, la vibración libre puede describirse mediante ecuaciones diferenciales ordinarias. Sin embargo, en aplicaciones prácticas la respuesta del sistema suele obtenerse mediante instrumentación, por ejemplo con acelerómetros, sensores de desplazamiento o registros de ensayos. Estos equipos entregan la información en forma de datos discretos; es decir, no se cuenta con una función continua del movimiento, sino con un conjunto finito de mediciones tomadas en instantes específicos. Esta forma de obtener los datos hace necesario utilizar métodos numéricos que permitan reconstruir, de manera aproximada, el movimiento continuo asociado al fenómeno observado [1, 2].

La interpolación atiende precisamente esta necesidad. A partir de un conjunto de pares ordenados de tiempo y respuesta, se construye una función aproximada que pasa por los puntos conocidos y que permite estimar la respuesta en instantes intermedios. Así, la interpolación conecta la información discreta medida con una representación continua del movimiento, lo cual facilita tareas posteriores como la estimación de velocidades, la identificación de máximos y mínimos, o el cálculo de cruces por cero [2].

Los métodos de interpolación, sin embargo, no presentan el mismo comportamiento frente a un mismo conjunto de datos. La interpolación polinómica global de Lagrange, los splines cúbicos definidos por tramos y la interpolación de Hermite, que incorpora información sobre las derivadas, poseen ventajas y limitaciones diferentes en términos de precisión, suavidad, estabilidad numérica y costo computacional. Comparar estos tres métodos resulta pertinente en el análisis de vibraciones, ya que la aproximación obtenida no solo debe ajustarse a los datos, sino también conservar una forma físicamente coherente con el carácter oscilatorio del movimiento [1, 5].

El objetivo de este trabajo es estudiar y comparar la interpolación de Lagrange, los splines cúbicos y la interpolación de Hermite como herramientas para aproximar el movimiento de vibración libre de un sistema dinámico a partir de una base de datos discreta. En este informe se presentan el desarrollo conceptual y la metodología del proyecto, mientras que todos los procedimientos numéricos se implementan y ejecutan en MATLAB.

2. Planteamiento del problema

Cuando se registra experimental o numéricamente el movimiento de vibración libre de un sistema dinámico, la información disponible se expresa como un conjunto finito de pares ordenados (t_i, x_i) , donde t_i corresponde al tiempo de medición y x_i a la respuesta del sistema, ya sea desplazamiento o aceleración, en ese instante. Esta representación discreta no permite evaluar directamente el movimiento en tiempos intermedios ni estudiar con suficiente detalle el comportamiento continuo del sistema.

Además, no siempre en un sismo se logran registrar todos los datos, ya sea por fallas en los instrumentos, o por la destrucción de los mismos. Por ejemplo, los acelerómetros necesitan cierta amplitud de movimiento para activarse, por lo que no se registran los datos previo a este momento [6]. Hay muchos otros casos como este, en el que hay una pérdida de datos, por lo que, es necesario tener alguna forma de conocer el comportamiento de un edificio, a partir de la información en ciertos puntos.

Por esta razón, se requiere reconstruir a partir de los datos discretos una función aproximada $p(t)$ que represente el movimiento dentro del intervalo de análisis. Existen distintos métodos de interpolación para realizar esta tarea, pero cada uno posee características propias. Por ello, no resulta inmediato determinar cuál ofrece la representación más adecuada del movimiento de vibración libre para una base de datos específica.

El problema abordado en este trabajo puede formularse de la siguiente manera: ¿cuál de los métodos de interpolación —Lagrange, splines cúbicos o Hermite— representa con mayor fidelidad y estabilidad el movimiento de vibración libre de un sistema dinámico a partir de un conjunto de datos discretos, considerando criterios de error, suavidad, estabilidad numérica, costo computacional e interpretabilidad física?

3. Objetivos

3.1. Objetivo general

Aproximar el movimiento de vibración libre de un sistema dinámico en MATLAB, con los métodos de interpolación de Lagrange, splines cúbicos e interpolación de Hermite.

3.2. Objetivos específicos

1. Explicar el fundamento teórico de la vibración libre en sistemas dinámicos de un grado de libertad y su representación mediante datos discretos.
2. Formular en MATLAB los métodos de interpolación de Lagrange, splines cúbicos e interpolación de Hermite a partir de los datos disponibles del proyecto.
3. Criticar las aproximaciones obtenidas mediante gráficas, métricas de error y criterios de fidelidad, para determinar cuál método representa mejor el movimiento de vibración libre del

sistema analizado.

4. Marco teórico

4.1. Vibración libre en sistemas dinámicos

Un sistema dinámico de un grado de libertad puede modelarse mediante una masa m , un elemento de rigidez k y un mecanismo de amortiguamiento de coeficiente c . Cuando el sistema oscila sin la acción de fuerzas externas, su comportamiento se describe mediante la ecuación diferencial ordinaria de segundo orden [3, 4]:

$$m \ddot{x}(t) + c \dot{x}(t) + k x(t) = 0, \quad (1)$$

donde:

- m es la masa del sistema,
- c es el coeficiente de amortiguamiento,
- k es la rigidez del sistema,
- $x(t)$ es el desplazamiento,
- $\dot{x}(t)$ es la velocidad,
- $\ddot{x}(t)$ es la aceleración.

En el caso ideal sin amortiguamiento ($c = 0$), la ecuación (1) se reduce a:

$$m \ddot{x}(t) + k x(t) = 0, \quad (2)$$

cuya solución es un movimiento armónico simple caracterizado por la *frecuencia natural*:

$$\omega_n = \sqrt{\frac{k}{m}}. \quad (3)$$

Cuando existe amortiguamiento, conviene introducir la *razón de amortiguamiento*:

$$\zeta = \frac{c}{2\sqrt{k m}}, \quad (4)$$

de modo que, en el régimen subamortiguado ($0 < \zeta < 1$), la respuesta de vibración libre adopta la forma de una oscilación que decae exponencialmente [4]:

$$x(t) = e^{-\zeta \omega_n t} [A \cos(\omega_d t) + B \sin(\omega_d t)], \quad \omega_d = \omega_n \sqrt{1 - \zeta^2}. \quad (5)$$

Aunque el modelo teórico descrito por las ecuaciones anteriores permite caracterizar el movimiento de forma analítica, en la práctica la respuesta del sistema se conoce a través de mediciones

discretas. Por ello, es necesario recurrir a métodos numéricos para reconstruir el movimiento continuo a partir de tales datos [1, 2].

4.2. Representación discreta del movimiento

La base de datos del proyecto está conformada por un conjunto finito de pares ordenados:

$$(t_i, x_i), \quad i = 0, 1, \dots, n, \quad (6)$$

donde t_i representa el tiempo y x_i la respuesta del sistema en ese instante (desplazamiento o, según el caso, aceleración).

El objetivo numérico consiste en construir una función aproximada:

$$p(t) \approx x(t), \quad (7)$$

que represente el movimiento del sistema en el intervalo de análisis y permita estimar la respuesta en instantes no medidos. La calidad de esta aproximación depende del método de interpolación empleado, de la distribución de los nodos y del nivel de ruido presente en los datos.

4.3. Interpolación polinómica de Lagrange

La interpolación de Lagrange construye un único polinomio de grado a lo sumo n que pasa exactamente por los $n + 1$ puntos conocidos [1, 2]. El polinomio interpolante se expresa como:

$$P_n(t) = \sum_{i=0}^n x_i L_i(t), \quad (8)$$

donde $L_i(t)$ son los polinomios base de Lagrange, definidos como:

$$L_i(t) = \prod_{\substack{j=0 \\ j \neq i}}^n \frac{t - t_j}{t_i - t_j}. \quad (9)$$

Cada polinomio base satisface la propiedad $L_i(t_j) = \delta_{ij}$, de modo que $P_n(t_i) = x_i$ en todos los nodos. El error de interpolación, cuando la función subyacente es suficientemente suave, puede acotarse mediante una expresión que involucra la $(n + 1)$ -ésima derivada de la función [2].

Entre sus ventajas se cuenta su formulación directa y su exactitud en los nodos. Sin embargo, la interpolación polinómica global presenta una limitación importante: al aumentar el número de nodos, especialmente si están igualmente espaciados, el polinomio puede desarrollar oscilaciones espurias de gran amplitud cerca de los extremos del intervalo, fenómeno conocido como *fenómeno de Runge* [1]. Esta inestabilidad puede distorsionar la representación de un movimiento de vibración libre cuando se emplean muchos puntos.

4.4. Splines cúbicos

Los splines cúbicos evitan las oscilaciones del polinomio global aproximando la función por tramos mediante polinomios de grado tres. En cada subintervalo $[t_i, t_{i+1}]$ se define un polinomio de la forma [2, 5]:

$$S_i(t) = a_i + b_i(t - t_i) + c_i(t - t_i)^2 + d_i(t - t_i)^3, \quad t_i \leq t \leq t_{i+1}. \quad (10)$$

Los coeficientes a_i, b_i, c_i, d_i se determinan imponiendo condiciones de continuidad en los nodos interiores:

- **Continuidad de la función:** el spline debe ser continuo, es decir, los tramos coinciden en los nodos.
- **Continuidad de la primera derivada:** las pendientes de los tramos adyacentes coinciden en los nodos.
- **Continuidad de la segunda derivada:** la curvatura es continua a través de los nodos.

Estas condiciones, junto con condiciones de frontera adecuadas (por ejemplo, spline natural, en el que la segunda derivada se anula en los extremos, o spline sujeto, en el que se imponen las pendientes en los extremos), conducen a un sistema lineal que define los coeficientes. El resultado es una curva suave de clase C^2 que, en general, es más estable que un único polinomio global de alto grado [5], lo que la hace especialmente adecuada para representar señales oscilatorias como las de vibración libre.

4.5. Interpolación de Hermite

La interpolación de Hermite utiliza no solo los valores de la función en los nodos, sino también los valores de su derivada [2]. Si en cada nodo se conocen o se aproximan las velocidades $\dot{x}_i$, es posible construir interpolantes que respeten simultáneamente la posición y la pendiente del movimiento. Esto resulta físicamente significativo en el estudio de vibraciones, donde la velocidad tiene un sentido cinemático directo.

En el intervalo $[t_i, t_{i+1}]$, el interpolante cúbico de Hermite puede expresarse mediante funciones base como:

$$H_i(t) = h_{00}(s) x_i + h_{10}(s) (t_{i+1} - t_i) \dot{x}_i + h_{01}(s) x_{i+1} + h_{11}(s) (t_{i+1} - t_i) \dot{x}_{i+1}, \quad (11)$$

donde el parámetro normalizado es:

$$s = \frac{t - t_i}{t_{i+1} - t_i}, \quad (12)$$

y las funciones base de Hermite son:

$$h_{00}(s) = 2s^3 - 3s^2 + 1, \quad (13)$$

$$h_{10}(s) = s^3 - 2s^2 + s, \quad (14)$$

$$h_{01}(s) = -2s^3 + 3s^2, \quad (15)$$

$$h_{11}(s) = s^3 - s^2. \quad (16)$$

Si las derivadas $\dot{x}_i$ no están disponibles directamente en la base de datos, pueden aproximarse numéricamente mediante diferencias finitas a partir de los valores de x_i y t_i [1]. La interpolación de Hermite produce una curva suave que conserva la información de la pendiente, lo que permite representar de manera coherente los tramos crecientes y decrecientes del movimiento oscilatorio.

4.6. Criterios de comparación entre métodos

Para comparar los tres métodos de interpolación se proponen los siguientes criterios [1, 2]:

- **Error respecto a los datos conocidos:** discrepancia entre la aproximación y los valores medidos (por ejemplo, error cuadrático medio o error máximo absoluto), evaluada en puntos de control.
- **Suavidad de la curva:** grado de continuidad y ausencia de oscilaciones artificiales en la representación.
- **Estabilidad numérica:** sensibilidad del método al número de nodos, a su distribución y a la presencia de ruido en los datos.
- **Representación de características físicas:** capacidad de reproducir máximos, mínimos y cruces por cero del movimiento.
- **Costo computacional:** esfuerzo requerido para construir y evaluar el interpolante.
- **Interpretabilidad física:** coherencia del resultado con el comportamiento esperado de un sistema en vibración libre.

4.7. Método de comparación de resultados por análisis MAE y MSRT

- **MAE (Error absoluto medio):** representa el promedio de los errores absolutos entre los valores obtenidos y los reales. En esta métrica, todos los errores pesan lo mismo, sin importar su tamaño. Para calcularlo, se utiliza la siguiente fórmula [7]:

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| \quad (17)$$

- **MSRT (Raíz del error cuadrático medio):** representa la raíz cuadrada del promedio de los errores al cuadrado. Al elevar al cuadrado cada error absoluto, esta métrica está hecha para castigar los errores grandes. Para calcularlo, se utiliza la siguiente fórmula [7]:

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2} \quad (18)$$

5. Metodología

5.1. Descripción general de la base de datos

Con el fin de lograr la replicación del experimento así como la veracidad del mismo, se pone a disposición del lector el acceso a los datos, instrucciones y todo aquel documento que resulte relevante.

La base de datos del proyecto está disponible en el repositorio público de GitHub:

https://github.com/adrianix360/respositorio_proyecto_analisis

Dicha base contiene los registros del movimiento de vibración libre del sistema analizado, organizados como pares ordenados de tiempo y respuesta, conforme a la expresión (6). La descripción detallada del formato, las unidades y la cantidad de registros se completará una vez revisado el contenido del repositorio.

5.2. Preparación de los datos

Antes de aplicar los métodos de interpolación, los datos se cargan y se organizan en MATLAB. Esta etapa comprende:

1. Carga de la base de datos desde el repositorio de GitHub.
2. Identificación de las variables relevantes (tiempo y desplazamiento, o tiempo y aceleración, según el contenido de la base).
3. Limpieza y ordenamiento de los datos, debido a que los datos utilizados presentan múltiples movimientos que producen vibración, solo se utiliza la primera vibración presente en la toma de datos.
4. Selección del intervalo de análisis y, cuando corresponda, de los nodos que se emplearán para construir las ecuaciones de interpolador.

5.3. Aplicación de la interpolación de Lagrange

Se construye el polinomio interpolante de Lagrange definido por las ecuaciones (8) y (9) a partir de los nodos seleccionados. La construcción de los polinomios base, el ensamblaje del polinomio interpolante y su evaluación sobre una malla fina de tiempo se realizan en MATLAB.

5.4. Aplicación de splines cúbicos

Se determinan los coeficientes de los splines cúbicos definidos por la ecuación (10), imponiendo las condiciones de continuidad descritas en el marco teórico y las condiciones de frontera seleccionadas. El planteamiento y la resolución del sistema lineal correspondiente, así como la evaluación del spline resultante, se efectúan en MATLAB.

5.5. Aplicación de la interpolación de Hermite

Se construye el interpolante cúbico de Hermite según las ecuaciones (11)–(16). Cuando las derivadas $\dot{x}_i$ no estén disponibles en la base de datos, se aproximan mediante diferencias finitas. La estimación de las derivadas, el ensamblaje del interpolante y su evaluación se llevan a cabo en MATLAB.

5.6. Comparación de resultados

Una vez obtenidas las tres aproximaciones, se comparan entre sí y respecto a los datos originales empleando los criterios definidos en el marco teórico. Esta etapa incluye:

1. Generación de gráficas comparativas de las aproximaciones frente a los datos medidos.
2. Cálculo de métricas de error y de otros criterios de comparación.
3. Discusión del método que representa con mayor fidelidad el movimiento de vibración libre.

5.7. Implementación computacional en MATLAB

Todos los procedimientos numéricos del proyecto se implementan mediante scripts en MATLAB, tomando como entrada los datos disponibles en el repositorio de GitHub. En particular, MATLAB se encarga de:

1. La carga y el procesamiento de la base de datos.
2. La selección de los nodos de interpolación.
3. La construcción de los interpolantes de Lagrange, splines cúbicos y Hermite.
4. El cálculo de los coeficientes correspondientes.
5. La evaluación de cada método sobre el intervalo de análisis.
6. La generación de las gráficas.
7. El cálculo de los errores y las métricas comparativas.
8. La comparación entre los tres métodos.

De esta manera, el documento LaTeX no contiene desarrollos numéricos extensos ni cálculos manuales paso a paso. Los resultados numéricos, gráficas y tablas comparativas se generan en MATLAB y posteriormente se incorporan a este documento en los espacios reservados. Los scripts utilizados se adjuntarán en el repositorio de GitHub del proyecto.

6. Resultados esperados

Se espera obtener, para cada método de interpolación, una aproximación del movimiento de vibración libre del sistema en el intervalo de análisis. A partir de las gráficas y de las métricas de error generadas en MATLAB, se prevé:

- Que la interpolación de Lagrange ajuste exactamente los nodos pero pueda presentar oscilaciones espurias al aumentar el número de puntos.
- Que los splines cúbicos produzcan una curva suave y estable, adecuada para representar el carácter oscilatorio del movimiento.
- Que la interpolación de Hermite, al incorporar la información de las derivadas, reproduzca de forma coherente la pendiente del movimiento.

La comparación cuantitativa y cualitativa permitirá identificar el método más adecuado para representar el movimiento de vibración libre del sistema analizado.

A continuación se adjunta la gráfica obtenida con los datos disponibles en el repositorio. Este gráfico se utilizará para comparar la ecuación teórica que se obtendrá por medio del análisis de algunos valores (t_i, x_i)

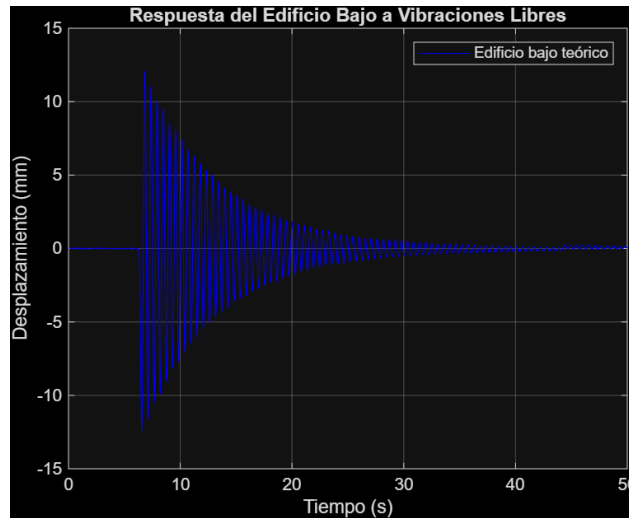


Figura 1: Gráfico resultado de los datos obtenidos por la mesa vibratoria

7. Resultados obtenidos

7.1. Ecuación teórica

Por medio del siguiente código se obtuvieron los valores necesarios para formular una ecuación que describe la vibración libre, la cual se graficó y comparó con la ecuación obtenida en la figura 1 Entradas:

T_d =Periodo natural amortiguado [s]

ξ =Índice de amortiguamiento

ω_n =Frecuencia circular natural [rad/s]

ω_d =Frecuencia circular amortiguada [rad/s]

```
1 function [A1,xi,omega_n,omega_d,T_n] = obtenerModelo(t,y)%funcion generica
   que recibe un vector tiempo y un vector desplazamiento y devuelve
   todas las variables que estan en la llave
2
3
4 [picos,local] = findpeaks(y,t,'MinPeakHeight',0.5);%localiza las tres
   primeras crestas para obtener un valor del periodo promedio
5
6 primeros_tiempos = local(1:3);
7
8 T1 = primeros_tiempos(2)-primeros_tiempos(1);
9 T2 = primeros_tiempos(3)-primeros_tiempos(2);
10
11 T_d = mean([T1 T2])%periodo natural promedio
12
13 % Frecuencia amortiguada
14 omega_d = 2*pi/T_d
15
16
17 [A1,idx] = max(y);
18 Amp_max=A1%amplitud maxima
19 tmax = t(idx);
20
21 t_objetivo = tmax + T_n;%periodo T+1
22
23 [~,idx2] = min(abs(t-t_objetivo));
24
25 A2 = y(idx2);%segunda cresta
26
27 % Decremento logaritmico
28 delta = log(A1/A2)
29
```

```

30 xi = delta/sqrt(4*pi^2 + delta^2)
31
32 % Frecuencia natural
33 omega_n = omega_d/sqrt(1-xi^2)
34
35 end

```

Con los datos anteriores se construye por medio del siguiente código la ecuación experimental:

$$12.1008e^{-0.0139 \cdot 11.4251 \cdot t} \text{sen}(11.4240t) \quad (19)$$

```

1 function x = vibracion(t,A,omega_n,omega_d,xi)%creacion de la ecuacion
   teorica a partir de los datos experimentales
2
3 x = A.*exp(-xi.*omega_n.*t).*sin(omega_d.*t);
4
5 end
6
7 t = libre(:,1);%vector de tiempo (s)
8 y = libre(:,3);%vector de desplazamiento del edificio bajo (mm)
9
10 %t = libre(:,1);%vector de tiempo (s)
11 %y = libre(:,4);%vector de desplazamiento del edificio alto (mm)
12
13 [A1,xi,omega_n,omega_d,T_d] = obtenerModelo(t,y);%calcula variables
14
15 y_modelo = vibracion(t,A1,omega_n,omega_d,xi);%crea la curva verde

```

Con el siguiente código se colocan ambas gráficas (la original con los datos medidos y la propuesta por la ecuación experimental):

```

1 figure
2 plot(t(1:3000),y(1:3000),'-b')
3 hold on
4 plot(t(1:3000),vibracion(t(1:3000),A1,omega_n,omega_d,xi),'-g')
5 hold off
6 legend('Edificio_bajo_teorico','Edificio_bajo_experimental')
7 xlabel('Tiempo(s)')
8 ylabel('Desplazamiento(mm)')
9 title('Respuesta_del_Edificio_Bajo_a_Vibraciones_Libres');
10 grid on;

```

En el siguiente gráfico la gráfica verde es el resultado de la ecuación experimental y la gráfica azul corresponde a los (t_i, x_i) medidos en campo. Es importante aclarar que se observa un desfase debido a una posible excitación del edificio antes de comenzar la medición de sus datos, por lo que

ambas curvas no presentan el mismo inicio.

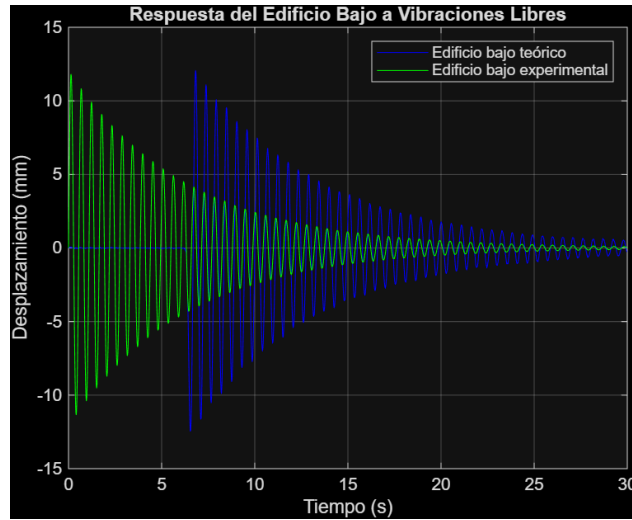


Figura 2: Gráfico experimental vs gráfico teórico

7.2. Polinomio interpolador de Hermite

Para poder encontrar el polinomio interpolador de Hermite, primero se procedió a realizar una limpieza de datos para que, posteriormente, se puedan encontrar las derivadas de la posición con respecto al tiempo. Estas se obtuvieron mediante el diferencias finitas adelante en el primer punto, atrás en el ultimo punto y centrada en los puntos interiores. Luego, utilizando la ecuaciones (11)-(16), se calculó el polinomio interpolador de Hermite. Las siguientes son las variables utilizadas en el código de Matlab:

Entradas:

t: vector de tiempos

x: vector de respuesta del sistema (desplazamiento, etc.)

npuntos: cantidad de puntos de la malla fina de evaluacion

Salidas:

tfino: malla fina de tiempo

xhermite: valores interpolados por Hermite

dxdt: derivadas aproximadas en los nodos

```
1 function [t_fino, x_hermite, dxdt] = hermite_vibracion(t, x, n_puntos)
2 % HERMITE_VIBRACION Interpolacion cubica de Hermite por tramos.
3
4     t = t(:);
5     x = x(:);
6
7     % --- Ordenar por tiempo ---
8     [t, idx] = sort(t);
```



```

9      x = x(idx);
10
11     % --- Eliminar tiempos repetidos (y advertir) ---
12     n_antes = length(t);
13     [t, iu] = unique(t);      % t queda ordenado y sin repeticiones
14     x = x(iu);
15     if length(t) < n_antes
16         warning('hermite_vibracion:duplicados', ...
17             'Se eliminaron %d valores de tiempo repetidos.', n_antes -
18                 length(t));
19     end
20
21     n = length(t);
22     if n < 2
23         error('Se necesitan al menos 2 puntos para interpolar.');
```

24

```

25     % --- Derivadas numericas en los nodos (diferencias finitas) ---
26     dxdt = zeros(n, 1);
27     dxdt(1) = (x(2) - x(1)) / (t(2) - t(1));      % adelante
28     dxdt(n) = (x(n) - x(n-1)) / (t(n) - t(n-1)); % atras
29     for i = 2:n-1
30         dxdt(i) = (x(i+1) - x(i-1)) / (t(i+1) - t(i-1)); % centrada
31     end
32
33     % --- Malla fina de evaluacion ---
34     t_fino = linspace(t(1), t(n), n_puntos)';
35     x_hermite = zeros(size(t_fino));
36
37     % --- Evaluacion del interpolante de Hermite por tramos ---
38     for k = 1:n-1
39         ti = t(k);
40         ti1 = t(k+1);
41         hi = ti1 - ti;      % h_i = t_{i+1} - t_i
42
43         % Puntos de la malla que caen en el intervalo [ti, ti1]
44         if k < n-1
45             sel = find(t_fino >= ti & t_fino < ti1);
46         else
47             sel = find(t_fino >= ti & t_fino <= ti1); % incluye el
48                 extremo final
49         end
50
51         s = (t_fino(sel) - ti) / hi;      % s = (t - t_i) / h_i

```

```

52         % Funciones base de Hermite
53         h00 = 2*s.^3 - 3*s.^2 + 1;
54         h10 = s.^3 - 2*s.^2 + s;
55         h01 = -2*s.^3 + 3*s.^2;
56         h11 = s.^3 - s.^2;
57
58         % H_i(t) = h00*x_i + h10*h_i*x'_i + h01*x_{i+1} + h11*h_i*x'_{i+1}
59         x_hermite(sel) = h00*x(k) + h10*hi*dxdt(k) ...
60                         + h01*x(k+1) + h11*hi*dxdt(k+1);
61     end
62 end

```

Finalmente, se decidió utilizar 3000 nodos para la construcción del polinomio interpolador debido a varias razones. La precisión que se obtuvo con esta cantidad de puntos es significativamente alta, por lo que el uso de mayor cantidad de nodos solo resulta en un costo computacional mayor sin ningún beneficio a cambio. Por lo tanto, su reducido costo computacional y la suavidad en la curva del interpolador obtenido justifican la cantidad de nodos implementados. A continuación se presenta la gráfica del polinomio en contraste con el movimiento de la estructura.

```

1 libre = readmatrix('Datos/vibracion_libre.txt');
2
3 t = libre(:,1); % tiempo (s)
4 y = libre(:,3); % desplazamiento (mm)
5
6 % Opcional: usar solo el primer tramo, como en el script base del proyecto
7
8
9 %% Calculo de derivadas numericas y evaluacion del interpolante
10 n_puntos = 3000;
11
12 % Resolucion de la malla fina
13 [t_fino, x_hermite, dxdt] = hermite_vibracion(t, y, n_puntos);
14
15 %% Grafica de resultados
16 figure
17 plot(t, y, '-b') % datos originales
18 hold on
19 plot(t_fino, x_hermite, '-g', 'LineWidth', 1.2) % interpolacion de
20 Hermite
21 hold off
22 legend('Datos_originales', 'Interpolacion_de_Hermite', 'Location', 'best')
23 xlabel('Tiempo_(s)')
24 ylabel('Desplazamiento_(mm)')
25 title('Interpolacion_de_Hermite_-_Vibracion_libre')

```

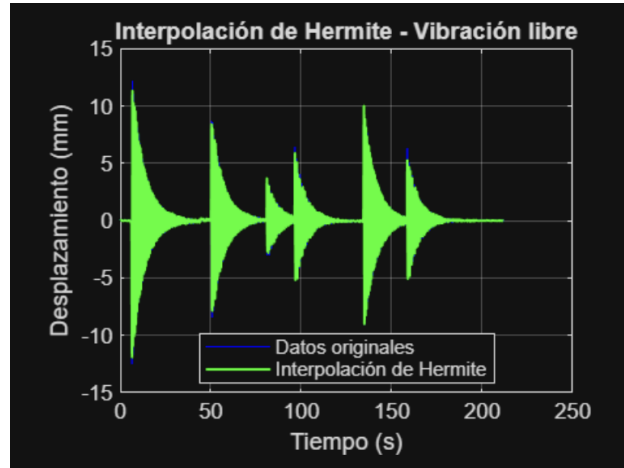


Figura 3: Comparación del polinomio interpolador de Hermite con el movimiento de la estructura

7.3. Resultados obtenidos con el método de splines cúbicos

El código de MATLAB para el método de splines cúbicos ordena primero los datos de tiempo y desplazamiento, elimina tiempos repetidos y calcula los anchos $h_i = t_{i+1} - t_i$ de cada intervalo. Luego construye un sistema tridiagonal para obtener los coeficientes c_i , asociados con la curvatura del spline. Este sistema se resuelve mediante el algoritmo de Thomas, que es eficiente para matrices tridiagonales. Finalmente, se calculan los coeficientes a_i , b_i y d_i , y se evalúa el spline en una malla fina de tiempo.

```

1  t = libre(:,1);
2  y = libre(:,3);
3
4
5  n_puntos = 3000;
6  [t_fino, x_spline, coeficientes] = spline_cubico_vibracion(t, y, n_puntos)
7  ;
8
9  %% Grafica de resultados
10 figure
11 plot(t, y, '-b')
12 hold on
13 plot(t_fino, x_spline, '-g', 'LineWidth', 1.2)
14 hold off
15 legend('Datos originales', 'Spline cubico natural', 'Location', 'best')
16 xlabel('Tiempo (s)')
17 ylabel('Desplazamiento (mm)')
```

```

18 title('Splines_cubicos-Vibracion_libre')
19 grid on
20
21 % --- Asegurar vectores columna ---
22 t = t(:);
23 x = x(:);
24
25 % --- Ordenar por tiempo ---
26 [t, idx] = sort(t);
27 x = x(idx);
28
29 % --- Eliminar tiempos repetidos (y advertir) ---
30 n_antes = length(t);
31 [t, iu] = unique(t);
32 x = x(iu);
33 if length(t) < n_antes
34     warning('spline_cubico:duplicados', ...
35         'Se eliminaron %d valores de tiempo repetidos.', n_antes -
36             length(t));
37
38 end
39
40 N = length(t);
41 if N < 3
42     error('Se necesitan al menos 3 puntos para un spline cubico.');
```

```

43 end
44
45 % --- Anchos de los intervalos ---
46 h = diff(t); % h(i) = t(i+1) - t(i), i = 1..N-1
47
48 % -----
49 % Construcción del sistema tridiagonal para los coeficientes c
50 % (c_i = S'(t_i)/2). Filas 1 y N imponen el spline natural.
51 % -----
52 sub = zeros(N,1); % subdiagonal (coef. de c_{i-1})
53 diag = zeros(N,1); % diagonal (coef. de c_i)
54 sup = zeros(N,1); % superdiagonal(coef. de c_{i+1})
55 d = zeros(N,1); % lado derecho
56
57 % Condiciones naturales: c_1 = 0 y c_N = 0
58 diag(1) = 1; d(1) = 0;
59 diag(N) = 1; d(N) = 0;
60
61 % Ecuaciones interiores i = 2..N-1
62 for i = 2:N-1
63     sub(i) = h(i-1);

```

```

62         diag(i) = 2*(h(i-1) + h(i));
63         sup(i) = h(i);
64         d(i) = 3*( (x(i+1)-x(i))/h(i) - (x(i)-x(i-1))/h(i-1) );
65     end
66
67     % --- Resolucion del sistema tridiagonal (algoritmo de Thomas) ---
68     % Eliminacion hacia adelante
69     for i = 2:N
70         w = sub(i)/diag(i-1);
71         diag(i) = diag(i) - w*sup(i-1);
72         d(i) = d(i) - w*d(i-1);
73     end
74     % Sustitucion hacia atras
75     c = zeros(N,1);
76     c(N) = d(N)/diag(N);
77     for i = N-1:-1:1
78         c(i) = (d(i) - sup(i)*c(i+1)) / diag(i);
79     end
80
81     % -----
82     % Calculo de coeficientes a, b, d de cada tramo i = 1..N-1
83     % -----
84     a = zeros(N-1,1);
85     b = zeros(N-1,1);
86     dd = zeros(N-1,1);
87     for i = 1:N-1
88         a(i) = x(i);
89         b(i) = (x(i+1)-x(i))/h(i) - h(i)/3*(2*c(i) + c(i+1));
90         dd(i) = (c(i+1) - c(i)) / (3*h(i));
91     end
92     c_tramo = c(1:N-1);
93
94     % --- Tabla de coeficientes ---
95     coeficientes = table((1:N-1)', t(1:N-1), t(2:N), a, b, c_tramo, dd,
96         ...
97         'VariableNames', {'i','t_i','t_ip1','a','b','c','d'});
98
99     % -----
100    % Evaluacion del spline en la malla fina
101    % -----
102    t_fino = linspace(t(1), t(N), n_puntos)';
103    x_spline = zeros(size(t_fino));
104
105    for k = 1:N-1
106        ti = t(k);

```

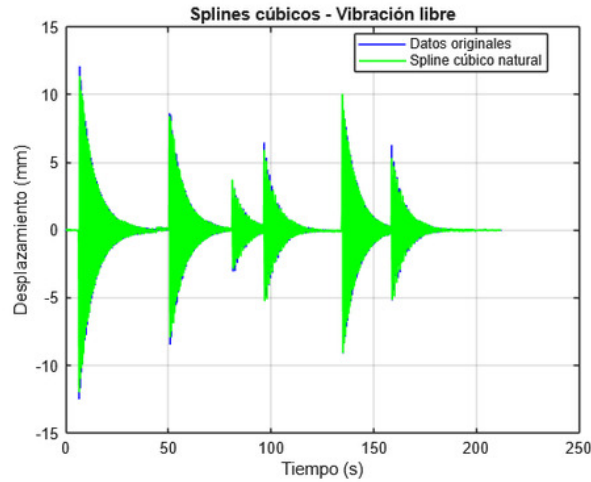


Figura 4: Comparación entre los datos originales de vibración libre y la aproximación mediante spline cúbico natural.

```

106     ti1 = t(k+1);
107
108     % Puntos de la malla que caen en el intervalo [ti, ti1]
109     if k < N-1
110         sel = find(t_fino >= ti & t_fino < ti1);
111     else
112         sel = find(t_fino >= ti & t_fino <= ti1);
113     end
114
115     dt = t_fino(sel) - ti;    % (t - t_i)
116     x_spline(sel) = a(k) + b(k)*dt + c_tramo(k)*dt.^2 + dd(k)*dt.^3;
117 end
118 end

```

El comportamiento obtenido con los splines cúbicos es adecuado para el análisis de vibraciones libres porque el método produce una curva continua y suave. Esta suavidad es relevante en ingeniería civil, ya que el desplazamiento estructural no debería representarse mediante una función con cambios bruscos o quiebres artificiales entre mediciones consecutivas. Al imponer continuidad de la función, de la primera derivada y de la segunda derivada, el spline genera una representación compatible con la idea física de un movimiento oscilatorio continuo.

La principal ventaja observada en los splines cúbicos es su estabilidad numérica. Como el método trabaja por tramos, cada polinomio cúbico solo representa una parte local de la señal. Esto evita que un error o una variación en una zona específica afecte de manera exagerada todo el intervalo. Esta característica marca una diferencia importante respecto al polinomio de Lagrange, ya que Lagrange construye un único polinomio global y, por lo tanto, puede amplificar oscilaciones artificiales, especialmente cerca de los extremos del intervalo.

En la comparación realizada, Lagrange tuvo un MAE de 13.049 mm y un RMSE de 33.477 mm, valores considerablemente superiores a los obtenidos por splines cúbicos. Esto confirma que, aunque Lagrange puede ser más rápido computacionalmente, no fue el método más adecuado para representar la señal de vibración libre. Su error máximo de 166.88 mm evidencia la aparición de discrepancias grandes, probablemente asociadas al fenómeno de Runge y a la dificultad de representar una señal oscilatoria completa mediante un solo polinomio.

i	t _i	t _{ip1}	a	b	c	d
1	0	0.01	-0.009357	-0.63529	0	-213.11
2	0.01	0.02	-0.015923	-0.69922	-6.3934	7631.6
3	0.02	0.03	-0.015923	1.4624	222.55	-11891
4	0.03	0.04	0.009065	2.3461	-134.18	4366
5	0.04	0.05	0.023474	0.97229	-3.2011	-3292.8
6	0.05	0.06	0.029584	-0.07957	-101.98	3607.2
7	0.06	0.07	0.022197	-1.0371	6.2308	15858
8	0.07	0.08	0.028307	3.8449	481.97	-46247

Figura 5: Primeros coeficientes calculados para los tramos del spline cúbico natural.

Por otro lado, Hermite obtuvo resultados ligeramente mejores que los splines cúbicos en las métricas de error. Esto se explica porque Hermite incorpora información de la pendiente, es decir, aproxima la derivada del desplazamiento con respecto al tiempo. En vibraciones, esa derivada está relacionada con la velocidad, por lo que el método puede representar de manera más ajustada los cambios de dirección de la señal. No obstante, esa ventaja también implica una dependencia adicional: si las derivadas no se conocen experimentalmente, deben aproximarse por diferencias finitas. Por esa razón, Hermite puede ser muy preciso, pero depende de la calidad de esas derivadas aproximadas.

Los splines cúbicos, en cambio, no requieren conocer directamente la velocidad ni aproximar derivadas externas en cada nodo. El método calcula la suavidad de la curva a partir de las condiciones internas del sistema tridiagonal. Esto lo convierte en una alternativa robusta y conveniente cuando se trabaja con datos experimentales de desplazamiento en función del tiempo, como ocurre en este proyecto.

Desde el punto de vista del costo computacional, el spline cúbico tuvo un tiempo de ejecución de 0.010803 s, mientras que Hermite tuvo 0.011669 s y Lagrange 0.008420 s. Aunque Lagrange fue el más rápido, su menor tiempo de ejecución no compensa el aumento significativo del error. En cambio, los splines cúbicos mantienen un costo computacional bajo y producen una aproximación mucho más estable y físicamente coherente.

En síntesis, los splines cúbicos representan un equilibrio favorable entre precisión, estabilidad y sentido físico. Aunque Hermite obtuvo el menor RMSE, la diferencia con splines fue pequeña. Además, el spline cúbico natural presentó una curva suave, evitó las oscilaciones grandes del polinomio global de Lagrange y permitió reconstruir adecuadamente el comportamiento oscilatorio del sistema. Por ello, para el análisis de vibraciones libres en ingeniería civil, los splines cúbicos pueden considerarse una herramienta numérica confiable para aproximar el movimiento continuo de una estructura

a partir de datos discretos.

7.4. Polinomio interpolador de Lagrange

Para obtener el polinomio interpolador de Lagrange, primero se extrajeron los valores de tiempo y desplazamiento del conjunto de datos. De esta base de datos, se seleccionan 15 puntos equiespaciados, cubriendo todo el espacio temporal para no concentrarse en una sola zona. El código ordena los puntos por tiempo, y garantiza que ninguno este repetido. Se deben usar pocos nodos para evitar el fenómeno de Runge. Obteniendo estos nodos iniciales, se crea un vector de 500 tiempos entre el primer y último nodo, para poder crear el polinomio y graficarlo en una línea continua. Para poder realizar este método, se utilizaron las siguientes variables:

Entradas:

t: vector de tiempos

x: vector de desplazamiento

num nodos: cantidad de puntos iniciales extraídos de la base de datos

n puntos: cantidad de puntos de la malla fina de evaluacion

Salidas:

t fino: vector de 500 tiempos equiespaciados

x lagrange: vector de 500 desplazamientos calculados por el polinomio de Lagrange

nodos t: vector con los 15 tiempos iniciales

nodos x: vector con los 15 desplazamientos correspondientes a los tiempos iniciales

```
1 % Carga de datos: columna 1 = tiempo, columna 3 = desplazamiento
2 t = libre(:,1);
3 y = libre(:,3);
4
5 % num_nodos controla el grado del polinomio (grado = num_nodos - 1).
6 % n_puntos define cuantos puntos tendra la curva interpolada al graficar.
7 num_nodos = 15;
8 n_puntos = 500;
9
10
11 [t_fino, x_lagrange, nodos_t, nodos_x] = lagrange_vibracion(t, y,
    num_nodos, n_puntos);
12
13 % Muestra una tabla con el indice, tiempo y desplazamiento de cada nodo
    seleccionado, util para verificar la distribucion.
14 nodos = table((1:length(nodos_t))', nodos_t, nodos_x, ...
15     'VariableNames', {'i','t_i','x_i'});
16 disp(nodos);
17
```



```

18
19 function [t_fino, x_lagrange, nodos_t, nodos_x] = lagrange_vibracion(t, x,
    num_nodos, n_puntos)
20
21 % Asegura vectores columna, ordena por tiempo y elimina duplicados
22 t = t(:);
23 x = x(:);
24 [t, idx] = sort(t);
25 x = x(idx);
26 n_antes = length(t);
27 [t, iu] = unique(t);
28 x = x(iu);
29 if length(t) < n_antes
30     warning('lagrange_vibracion:duplicados', ...
31         'Se eliminaron %d valores de tiempo repetidos.', n_antes -
            length(t));
32 end
33 N = length(t);
34
35 % Selecciona num_nodos puntos equiespaciados del dataset
36 if num_nodos < 2
37     num_nodos = 2;
38 end
39 if N <= num_nodos
40     sel = (1:N)';
41 else
42     sel = unique(round(linspace(1, N, num_nodos)))';
43 end
44 nodos_t = t(sel);
45 nodos_x = x(sel);
46 n = length(nodos_t);
47
48 % Avisa si el grado es alto (riesgo de oscilaciones de Runge)
49 if n > 15
50     warning('lagrange_vibracion:grado_alto', ...
51         ['Se usan %d nodos (grado %d). Grados altos pueden causar ',
52             ...
53             'oscilaciones y perdida de estabilidad numerica.'], n, n-1);
54 end
55
56 % Malla fina donde se evaluara el polinomio. El polinomio solo se
    evalua dentro de este rango.
57 t_fino = linspace(nodos_t(1), nodos_t(n), n_puntos)';
58 x_lagrange = zeros(size(t_fino));

```

```

59     % Construccion de P(t) = sum_i x_i * L_i(t)
60
61     for i = 1:n
62         Li = ones(size(t_fino));
63         for j = 1:n
64             if j ~= i
65                 Li = Li .* (t_fino - nodos_t(j)) / (nodos_t(i) - nodos_t(j));
66             end
67         end
68         x_lagrange = x_lagrange + nodos_x(i) * Li;
69     end
70 end

```

Tras haber obtenido estos datos, se puede graficar la función del polinomio interpolador de Lagrange. La razón principal por la que utilizaron 500 puntos es para poder obtener una curva suave y continua, pero sin un costo computacional excesivo. El siguiente código fue el que se utilizó para crear el gráfico, en el cual, los puntos azules representan los datos verdaderos medidos en el sismo, la curva verde es el polinomio interpolador obtenido, y los puntos rojos son los 15 nodos utilizados inicialmente para calcular el polinomio.

```

1  figure
2  plot(t, y, '-b')
3  hold on
4  plot(t_fino, x_lagrange, '-g', 'LineWidth', 1.2)
5  scatter(nodos_t, nodos_x, 40, 'r', 'filled')
6  hold off
7  legend('Datos originales', 'Interpolacion de Lagrange', 'Nodos', ...
8         'Location', 'best')
9  xlabel('Tiempo (s)')
10 ylabel('Desplazamiento (mm)')
11 title('Interpolacion de Lagrange - Vibracion libre')
12 grid on

```

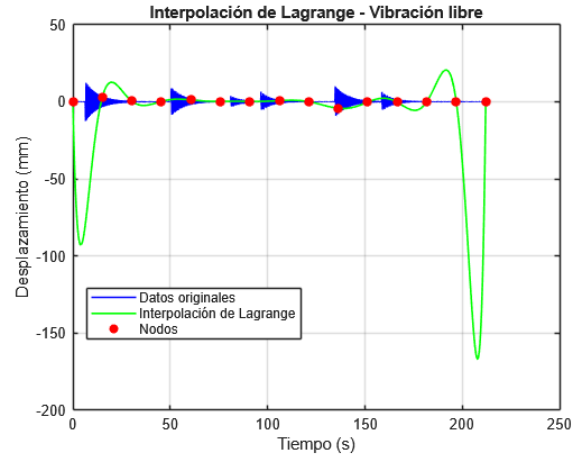


Figura 6: Comparación del polinomio interpolador de Lagrange con el movimiento de la estructura

8. Análisis de resultados

Después de haber calculado y resuelto los polinomios interpoladores de Hermite, Lagrange y de los splines cúbicos, es importante analizar la fidelidad de sus curvas con el movimiento de la estructura, el costo computacional y la estabilidad numérica. Por lo tanto, se calculó el error máximo absoluto y el error cuadrático medio y el error absoluto medio de cada método, con el fin de comparar la magnitud de sus discrepancias con la vibración entre sí. Por otro lado, se midió el tiempo que le toma al sistema computacional procesar y calcular cada método para contrastar los costos computacionales asociados. Para una comparación justa, se construyeron polinomios con el mismo número de nodos y se evalúan en el mismo vector de tiempo. A continuación se presenta el cálculo de los polinomios con la misma cantidad de nodos y los errores de cada método.

```

1  function metricas = calcular_metricas(x_real, x_estimado,
    tiempo_ejecucion)
2
3  x_real      = x_real(:);
4  x_estimado = x_estimado(:);
5  err = x_real - x_estimado;
6
7  MAE          = mean(abs(err));
8  RMSE         = sqrt(mean(err.^2));
9  ErrorMaximo  = max(abs(err));
10
11  SS_res = sum(err.^2);
12  SS_tot = sum((x_real - mean(x_real)).^2);
13  if SS_tot == 0
14      R2 = NaN;
15  else
16      R2 = 1 - SS_res/SS_tot;

```

```

17     end
18
19     metricas = [MAE, RMSE, ErrorMaximo, R2, tiempo_ejecucion];
20 end
21
22 function x_eval = lagrange_eval(nodos_t, nodos_x, t_eval)
23 % LAGRANGE_EVAL Polinomio de Lagrange (manual) evaluado en t_eval.
24     nodos_t = nodos_t(:); nodos_x = nodos_x(:); t_eval = t_eval(:);
25     n = length(nodos_t);
26     x_eval = zeros(size(t_eval));
27     for i = 1:n
28         Li = ones(size(t_eval));
29         for j = 1:n
30             if j ~= i
31                 Li = Li .* (t_eval - nodos_t(j)) / (nodos_t(i) - nodos_t(j));
32             end
33         end
34         x_eval = x_eval + nodos_x(i) * Li;
35     end
36 end
37
38 function x_eval = spline_eval(nodos_t, nodos_x, t_eval)
39 % SPLINE_EVAL Spline cubico natural (manual) evaluado en t_eval.
40     nt = nodos_t(:); nx = nodos_x(:); t_eval = t_eval(:);
41     N = length(nt);
42     h = diff(nt);
43
44     % Sistema tridiagonal para c (condiciones naturales c_1 = c_N = 0)
45     sub = zeros(N,1); dg = zeros(N,1); sup = zeros(N,1); rhs = zeros(N,1);
46     dg(1) = 1; dg(N) = 1;
47     for i = 2:N-1
48         sub(i) = h(i-1);
49         dg(i) = 2*(h(i-1) + h(i));
50         sup(i) = h(i);
51         rhs(i) = 3*( (nx(i+1)-nx(i))/h(i) - (nx(i)-nx(i-1))/h(i-1) );
52     end
53     % Thomas
54     for i = 2:N
55         w = sub(i)/dg(i-1);
56         dg(i) = dg(i) - w*sup(i-1);
57         rhs(i) = rhs(i) - w*rhs(i-1);
58     end
59     c = zeros(N,1);
60     c(N) = rhs(N)/dg(N);

```

```

61     for i = N-1:-1:1
62         c(i) = (rhs(i) - sup(i)*c(i+1)) / dg(i);
63     end
64
65     % Coeficientes por tramo
66     a = nx(1:N-1);
67     b = zeros(N-1,1); dcoef = zeros(N-1,1);
68     for i = 1:N-1
69         b(i) = (nx(i+1)-nx(i))/h(i) - h(i)/3*(2*c(i) + c(i+1));
70         dcoef(i) = (c(i+1) - c(i)) / (3*h(i));
71     end
72
73     % Evaluacion
74     x_eval = zeros(size(t_eval));
75     for k = 1:N-1
76         ti = nt(k); ti1 = nt(k+1);
77         if k < N-1
78             sel = find(t_eval >= ti & t_eval < ti1);
79         else
80             sel = find(t_eval >= ti & t_eval <= ti1);
81         end
82         dt = t_eval(sel) - ti;
83         x_eval(sel) = a(k) + b(k)*dt + c(k)*dt.^2 + dcoef(k)*dt.^3;
84     end
85 end
86
87 function x_eval = hermite_eval(nodos_t, nodos_x, t_eval)
88 % HERMITE_EVAL Hermite cubico por tramos (manual) evaluado en t_eval.
89     nt = nodos_t(:); nx = nodos_x(:); t_eval = t_eval(:);
90     n = length(nt);
91
92     % Derivadas por diferencias finitas en los nodos
93     d = zeros(n,1);
94     d(1) = (nx(2)-nx(1))/(nt(2)-nt(1));
95     d(n) = (nx(n)-nx(n-1))/(nt(n)-nt(n-1));
96     for i = 2:n-1
97         d(i) = (nx(i+1)-nx(i-1))/(nt(i+1)-nt(i-1));
98     end
99
100     % Evaluacion por tramos
101     x_eval = zeros(size(t_eval));
102     for k = 1:n-1
103         ti = nt(k); ti1 = nt(k+1); hi = ti1 - ti;
104         if k < n-1
105             sel = find(t_eval >= ti & t_eval < ti1);

```

```

106         else
107             sel = find(t_eval >= ti & t_eval <= ti1);
108         end
109         s = (t_eval(sel) - ti) / hi;
110         h00 = 2*s.^3 - 3*s.^2 + 1;
111         h10 = s.^3 - 2*s.^2 + s;
112         h01 = -2*s.^3 + 3*s.^2;
113         h11 = s.^3 - s.^2;
114         x_eval(sel) = h00*nx(k) + h10*hi*d(k) + h01*nx(k+1) + h11*hi*d(k
            +1);
115     end
116 end

```

Posteriormente, se realizó la comparación entre los métodos.

```

1  % Puntos comunes de comparacion
2  t_eval = t;           % comparar en los tiempos originales
3  x_real = y;           % datos de referencia
4
5  % Nodos comunes para los tres metodos
6  % num_nodos controla el grado del polinomio de Lagrange (grado = num_nodos
    -1)
7  % y la cantidad de tramos de spline/Hermite. Subirlo mejora spline/Hermite
8  % pero degrada Lagrange (oscilaciones de Runge).
9  num_nodos = 15;
10 N = length(t);
11 if N <= num_nodos
12     sel_nodos = (1:N)';
13 else
14     sel_nodos = unique(round(linspace(1, N, num_nodos)))';
15 end
16 nodos_t = t(sel_nodos);
17 nodos_x = y(sel_nodos);
18
19 % Evaluacion de cada metodo en t_eval (con medicion de tiempo)
20 tic;
21 x_lagrange_eval = lagrange_eval(nodos_t, nodos_x, t_eval);
22 tiempo_lagrange = toc;
23 % % Si Lagrange proviene de otra implementacion, insertar aqui
    x_lagrange_eval
24 % % evaluado en t_eval (mismos puntos que x_real).
25
26 tic;
27 x_spline_eval = spline_eval(nodos_t, nodos_x, t_eval);
28 tiempo_spline = toc;
29 % % Insertar aqui x_spline_eval evaluado en t_eval si viene de otra parte.

```

```

30
31 tic;
32 x_hermite_eval = hermite_eval(nodos_t, nodos_x, t_eval);
33 tiempo_hermite = toc;
34 % % Insertar aqui x_hermite_eval evaluado en t_eval si viene de otra parte
35
36 % Calculo de metricas
37 m_lagrange = calcular_metricas(x_real, x_lagrange_eval, tiempo_lagrange);
38 m_spline   = calcular_metricas(x_real, x_spline_eval, tiempo_spline);
39 m_hermite   = calcular_metricas(x_real, x_hermite_eval, tiempo_hermite);
40
41 M = [m_lagrange; m_spline; m_hermite];
42 Metodo      = ["Lagrange"; "Splines cubicos"; "Hermite"];
43 MAE          = M(:,1);
44 RMSE         = M(:,2);
45 ErrorMaximo  = M(:,3);
46 R2           = M(:,4);
47 TiempoEjecucion = M(:,5);
48
49 tabla_eficacia = table(Metodo, MAE, RMSE, ErrorMaximo, R2, TiempoEjecucion
50 );
51 disp(tabla_eficacia);

```

Metodo	MAE	RMSE	ErrorMaximo	R2	TiempoEjecucion
"Lagrange"	13.049	33.477	166.88	-343.73	0.00842
"Splines cubicos"	1.2797	2.2835	14.332	-0.60401	0.010803
"Hermite"	1.2366	2.2543	14.189	-0.56315	0.011669

Figura 7: Comparación de los errores y tiempos de ejecución de cada método

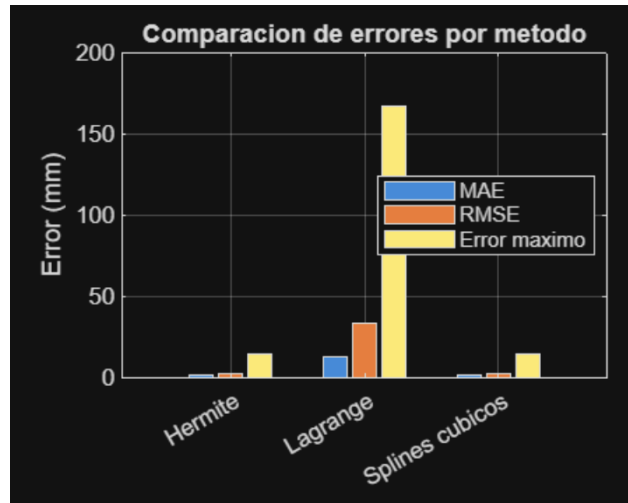


Figura 8: Comparación de los errores de cada método

Como se observa en la figura 5 y 6, el método de Hermite y de los splines cúbicos poseen un error muy bajo y muy similar entre ellos. En el método de Hermite el error bajo se debe a que el polinomio no solo retiene información sobre los nodos, sino que también de sus derivadas. De esta forma la interpolación representa de forma más realista los máximos, mínimos y cambios de dirección, manteniendo un error de aproximación pequeño incluso con un número moderado de puntos. El método de Hermite posee una buena estabilidad numérica y mitiga los efectos del ruido de los datos.

En cuanto al método de splines cúbicos, su baja discrepancia con los puntos de referencia se debe a su construcción por tramos. Los polinomios cúbicos son definidos localmente en cada tramo. La imposición de continuidad de la función y de sus dos primeras derivadas produce una curva suave y reduce las oscilaciones propias de la interpolación polinómica global. Además, en virtud de su construcción por segmentos, es un método numéricamente muy estable.

Un caso completamente distinto es el polinomio interpolador de Lagrange, en el que se obtuvo un error sumamente elevado en comparación con los otros dos métodos. Esto se debe ya que el método construye un único polinomio global de grado elevado para representar todo el movimiento. Esta condición lo hace susceptible al fenómeno de Runge, generando oscilaciones, especialmente cerca de los extremos del intervalo de interpolación. Además, al no incorporar información de las derivadas ni realizar una aproximación por tramos, su capacidad para representar funciones con variaciones locales es menor. El polinomio de Lagrange es el método numéricamente más inestable ya que, al modelar en un solo polinomio todo el movimiento, sufre mucho por el ruido y datos erróneos.

Finalmente, en la figura 5 visualizamos que se toma menos tiempo para construir el polinomio de Lagrange que los polinomios de Hermite y splines cúbicos. Por lo tanto, el costo computacional asociado al método de Lagrange es el más bajo de todos.

9. Comparación cuantitativa de los métodos

Cuadro 1: Métricas obtenidas en la comparación de métodos del archivo MATLAB Live Script.

Método	MAE (mm)	RMSE (mm)	Error máximo (mm)	R^2	Tiempo (s)
Lagrange	13.0490	33.4770	166.8800	-343.7300	0.008420
Splines cúbicos	1.2797	2.2835	14.3320	-0.60401	0.010803
Hermite	1.2366	2.2543	14.1890	-0.56315	0.011669

De acuerdo con la tabla 1, el método de Hermite presentó el menor RMSE, con un valor de 2.2543 mm. Sin embargo, el método de splines cúbicos obtuvo un valor muy cercano, con un RMSE de 2.2835 mm. La diferencia entre ambos métodos es de apenas 0.0292 mm, por lo que, en términos prácticos, ambos métodos ofrecen una aproximación muy similar para los datos analizados.

En el caso de los splines cúbicos, el error absoluto medio fue de 1.2797 mm y el error máximo fue de 14.332 mm. Estos resultados indican que, en promedio, la discrepancia entre el desplazamiento medido y el desplazamiento interpolado por splines es baja. El error máximo es mayor porque la comparación se realiza contra todos los datos originales, mientras que la curva se construye a partir de una representación interpolada que no necesariamente reproduce cada variación local o cada posible perturbación presente en los datos experimentales.

El valor de R^2 para los splines cúbicos fue negativo. Este resultado no debe interpretarse como que el método sea inútil, sino como una consecuencia de la forma en que se calculó la comparación: el modelo se construyó con nodos seleccionados y luego se comparó contra todos los datos medidos. Si la señal experimental tiene ruido, pequeñas irregularidades locales o zonas de baja amplitud alrededor del equilibrio, el indicador R^2 puede penalizar fuertemente las diferencias residuales. Por esta razón, para este proyecto conviene interpretar el desempeño de los métodos usando conjuntamente el MAE, el RMSE, el error máximo, la forma gráfica de la curva y la coherencia física del movimiento.

10. Conclusiones

En primer lugar, se logró describir el fundamento teórico de la vibración libre, así como su representación discreta, la cuál está compuesta por un conjunto de pares ordenados que representan el tiempo y la respuesta del sistema. Luego se formularon matemáticamente los métodos de interpolación de Hermite, splines cúbicos e interpolación de Lagrange, para poder implementarlos en MATLAB con los datos disponibles.

Después de hacer las gráficas comparativas de las aproximaciones, y contrastar las métricas de error, los resultados obtenidos coincidieron con lo esperado. A pesar de que el método de Lagrange tuvo un bajo costo computacional, fue el más inestable numéricamente, porque al ser solo un polinomio, se generaron oscilaciones en los extremos. El método de Hermite tuvo un costo computacional más elevado a comparación de Lagrange, ya que debe tomar en cuenta las derivadas en los nodos, pero esto lo hace tener un mejor desempeño, ya que puede representar mejor los cambios de dirección

que ocurren en la onda. Finalmente, el método de splines tuvo un costo computacional similar al de Hermite, pero al ser construido por tramo, se produjo una curva suave, y sin oscilaciones.

Basado en los errores calculados, Lagrange es el método menos adecuado a pesar de su bajo costo comutacional, ya que su MAE, RMSE, y su error máximo son considerablemente mayores que los de los otros métodos. Los valores obtenidos para el método de Hermite y splines son bastante similares, siendo respectivamente de 1.24 y 1.28 para MAE, 2.25 y 2.28 para RMSE, y 14.19 y 14.33 para el error máximo. Se podría concluir por los errores que el método de Hermite representa ligeramente con mayor fidelidad el movimiento de vibración libre. Sin embargo, obtener estas derivadas no siempre es tan sencillo, y puede ser necesario aproximarlas mediante diferencias finitas, introduciendo una independencia adicional, por lo tanto, finalmente se determina que el método más adecuado es el de splines.

Referencias

- [1] S. C. Chapra y R. P. Canale, *Métodos numéricos para ingenieros*, 7^a ed. México D.F., México: McGraw-Hill, 2015.
- [2] R. L. Burden, J. D. Faires y A. M. Burden, *Análisis numérico*, 10^a ed. Ciudad de México, México: Cengage Learning, 2017.
- [3] S. S. Rao, *Mechanical Vibrations*, 6th ed. Hoboken, NJ, USA: Pearson, 2017.
- [4] A. K. Chopra, *Dynamics of Structures: Theory and Applications to Earthquake Engineering*, 5th ed. Hoboken, NJ, USA: Pearson, 2017.
- [5] C. de Boor, *A Practical Guide to Splines*, rev. ed. New York, NY, USA: Springer, 2001.
- [6] L. Suárez, *Diferencia entre sismómetros y acelerómetros (y las redes de PR)*, Reporte técnico, Mayagüez, PR, USA: Universidad de Puerto Rico, Recinto Universitario de Mayagüez, mar. 2020.
- [7] T. Chai y R. R. Draxler, “Root mean square error (RMSE) or mean absolute error (MAE)? – Arguments against avoiding RMSE in the literature,” *Geosci. Model Dev.*, vol. 7, no. 3, pp. 1247–1250, 2014.
- [8] A. Quarteroni, R. Sacco y F. Saleri, *Numerical Mathematics*, 2nd ed. Berlin, Germany: Springer, 2007.